

CS-AML

Technology Architecture

Version 0.1.1

Approved Internal Specification Baseline

2026-10-08 · tag v0.1.1-spec

Civil Society Anti-Money Laundering & Financial Intelligence Framework

Generated from the canonical Markdown source. The Markdown file in the repository remains authoritative; change tags such as `[v0.1.x · ...]` are part of the traceability record.

Contents

Document Structure	5
1. Purpose, Scope, and Architectural Outcomes	6
1.1 Primary architectural goal	6
1.2 Required outcomes	7
1.3 Non-goals	7
2. Normative Architecture Principles	7
3. Stakeholders, Roles, and Trust Boundaries	8
3.1 Human roles	8
3.2 Trust boundaries	8
4. Capability Architecture	8
4.1 Crosswalk TA-CAP ↔ product CAP [v0.1.1 · A07]	9
5. Logical Architecture	9
5.1 Architectural rule: canonical store vs projections	10
6. Information and Data Architecture	10
6.1 Data planes	10
6.2 Storage responsibilities	10
6.3 Object identity and referential integrity	10
7. Case and Workflow Architecture	11
7.1 Workflow engine requirements	11
8. Evidence and Provenance Architecture	11
8.1 Evidence lifecycle	11
8.2 Original evidence requirements	11
8.3 Derivative chain	11
9. Entity Resolution Architecture	12
9.1 Resolution pipeline	12
9.2 Mandatory controls	12
9.3 Matching features	12
10. Knowledge Graph Architecture	13
10.1 Graph representation	13
10.2 Edge semantics	13
10.3 Supported graph operations	13
11. Search and Retrieval Architecture	13
11.1 Search modes	13
12. Timeline, Asset, and Value-Flow Analytics	14
12.1 Timeline service	14
12.2 Asset analysis	14
12.3 Value-flow engine	14
12.4 Flow composition	14
13. Typology and Analytical Services	14
13.1 Typology service model	14
13.2 Analytical jobs	15
14. Hypothesis, Assessment, and Intelligence Product Services	15
14.1 Hypothesis engine	15
14.2 Assessment service	15
14.3 Product generation	15

15. API and Integration Architecture	15
15.1 API principles	15
15.2 Minimum interfaces	16
15.3 Import and connector model	16
16. Identity, Authentication, and Authorization	16
16.1 Authentication	16
16.2 Authorization model	16
16.3 Authorization dimensions	17
16.4 Break-glass access	17
17. Privacy and Data-Protection Architecture	17
17.1 Classification model [v0.1.1 · A08]	17
18. Cryptography, Secrets, and Key Management	17
19. Security Architecture and Threat Model	18
19.1 High-value assets	18
19.2 Threat scenarios	18
19.3 Security control domains	18
20. Audit, Logging, and Non-Repudiation	19
20.1 Mandatory audit events	19
20.2 Audit integrity	19
21. Observability and Operational Monitoring	19
21.1 Signals	19
22. Reliability, Backup, and Disaster Recovery	19
22.1 Data criticality	19
22.2 Backup requirements	20
22.3 Example recovery profiles	20
23. Deployment Models	20
23.1 Model A - Single-node secure deployment	20
23.2 Model B - Segmented self-hosted deployment	20
23.3 Model C - Managed cloud deployment	20
23.4 Model D - High-risk isolated deployment	21
24. Multi-Tenancy and Federation	21
24.1 Multi-tenancy	21
24.2 Federation	21
24.3 Federation package requirements	21
25. Software Supply Chain and Secure Delivery	21
26. AI and Automation Architecture	22
26.1 AI service boundary	22
26.2 Model governance	22
27. Performance and Scalability	22
27.1 Performance classes	22
27.2 Scaling order	22
28. Reference Implementation Profiles	23
28.1 Profile R1 - Compact open-source deployment	23
28.2 Profile R2 - Segmented production deployment	23
28.3 Technology selection criteria	23
29. Architecture Decision Records	24

30. Conformance and Verification **24**

 30.1 Technology conformance levels 24

 30.2 Mandatory verification questions 24

 30.3 Architecture evidence pack 25

Annex A. Normative Component Catalogue **25**

Annex B. Minimum Network Exposure Baseline **25**

Annex C. Architecture Review Checklist **26**

Annex D. Reference Standards and External Lineage **26**

Annex E. Example End-to-End Technical Trace **26**

Document Status and Change Control **27**

CS-AML
Technology Architecture
Version 0.1.1

Document status — v0.1.1
Version: 0.1.1 — Approved Internal Specification Baseline (2026-10-08, tag v0.1.1-spec). [v0.1.1 · A01]
Supersedes: CS-AML Technology Architecture v0.1. The DOCX/PDF files in this repository are the unchanged v0.1 baseline (legacy); this Markdown file is the canonical source.
Validation: approved by the product owner as the internal specification baseline on 2026-10-08 (decision register and release gates in CHANGELOG.md). No implementation test result or independent audit exists yet. Acceptance criteria in this document are targets, not evidence that tests have passed.
CS-AML is not an external standard or certification. References to FATF, Wolfsberg, PPATK, UNODC or other bodies do not imply their endorsement.
Changes in 0.1.1: see CHANGELOG.md at the repository root (audit findings A01–A16).

Derivative specification — proposed normative baseline (draft for review) [v0.1.1 · A01]

Civil Society Financial Intelligence / AML Investigation Framework

Architectural axiom
Technology SHALL preserve analytical uncertainty rather than erase it. Direct evidence, reconstructed value flow, inference, hypothesis, and assessment MUST remain distinguishable in storage, API responses, search, visualisation, automation, and exported intelligence products.

Document field	Value
Document ID	CSAML-TA-0.1
Version	0.1.1
Status	Approved Internal Specification Baseline (2026-10-08, tag v0.1.1-spec) [v0.1.1 · A01]
Applies to	Systems implementing CS-AML investigations and intelligence workflows
Dependency	CS-AML Framework v0.1.1 (Expanded), Investigation Methodology v0.1.1, Data Model Specification v0.1.1, Control Implementation Guide v0.1.1, Typology Catalogue v0.1.1 (Markdown, Documents/*_v0.1.1*.md) [v0.1.1 · A01]
Normative language	SHALL / MUST = mandatory; SHOULD = recommended; MAY = optional
Architecture stance	Implementation-neutral core with non-normative reference profiles

This document specifies the minimum architectural capabilities, trust boundaries, service responsibilities, data planes, security controls, interoperability rules, and deployment characteristics required for a technology system to support CS-AML in a defensible and auditable manner.

Document Structure

- 1. Purpose, Scope, and Architectural Outcomes
- 2. Normative Architecture Principles
- 3. Stakeholders, Roles, and Trust Boundaries
- 4. Capability Architecture
- 5. Logical Architecture
- 6. Information and Data Architecture

- 7. Case and Workflow Architecture
- 8. Evidence and Provenance Architecture
- 9. Entity Resolution Architecture
- 10. Knowledge Graph Architecture
- 11. Search and Retrieval Architecture
- 12. Timeline, Asset, and Value-Flow Analytics
- 13. Typology and Analytical Services
- 14. Hypothesis, Assessment, and Intelligence Product Services
- 15. API and Integration Architecture
- 16. Identity, Authentication, and Authorization
- 17. Privacy and Data-Protection Architecture
- 18. Cryptography, Secrets, and Key Management
- 19. Security Architecture and Threat Model
- 20. Audit, Logging, and Non-Repudiation
- 21. Observability and Operational Monitoring
- 22. Reliability, Backup, and Disaster Recovery
- 23. Deployment Models
- 24. Multi-Tenancy and Federation
- 25. Software Supply Chain and Secure Delivery
- 26. AI and Automation Architecture
- 27. Performance and Scalability
- 28. Reference Implementation Profiles
- 29. Architecture Decision Records
- 30. Conformance and Verification
- Annexes

1. Purpose, Scope, and Architectural Outcomes

The Technology Architecture translates the CS-AML normative methodology, canonical data model, and control catalogue into a system architecture that can be implemented by civil-society organisations, investigative networks, research teams, or trusted technical service providers. It does not prescribe one software product or vendor.

1.1 Primary architectural goal

Primary goal

Provide a secure, evidence-preserving, analyst-centred technology environment that supports lawful collection, structured investigation, entity and relationship analysis, follow-the-value reconstruction, hypothesis testing, review, and controlled dissemination without converting uncertainty into false certainty.

1.2 Required outcomes

- Every material analytical statement can be traced to source and evidence objects.
- Analytical objects remain versioned and attributable to human or system actors.
- Direct, documented, reconstructed, and hypothetical value flows remain visibly distinct.
- Entity resolution decisions are reversible, reviewable, and supported by evidence.
- High-impact actions require appropriate human authorization and produce durable audit evidence.
- Privacy classification and access restrictions travel with data objects rather than relying only on folder or network location.
- The system can be operated in resource-constrained civil-society environments without requiring hyperscale infrastructure.
- The architecture supports incremental maturity: a small deployment can conform without implementing every advanced analytical component.

1.3 Non-goals

- Automated determination of criminal liability.
- Mass population surveillance or indiscriminate financial profiling.
- Unauthorized access to banking or communications systems.
- Replacement of FIU, regulator, law-enforcement, prosecutorial, or judicial functions.
- Black-box risk scores that cannot be explained or challenged.
- A mandatory microservices architecture or mandatory graph database.

2. Normative Architecture Principles

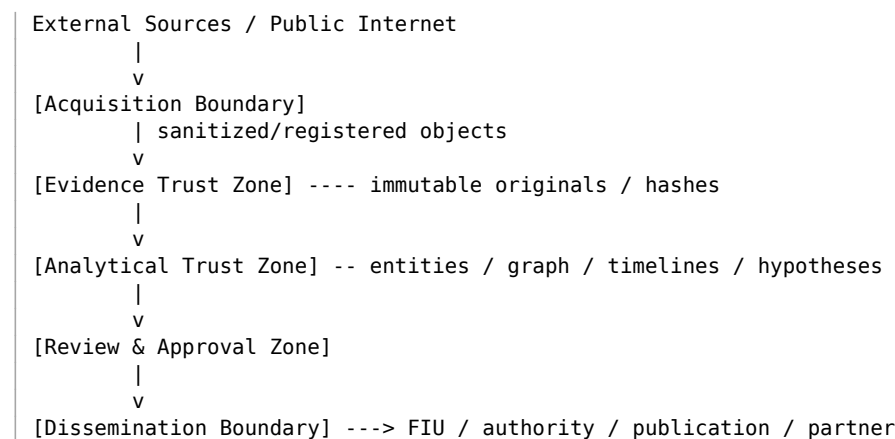
ID	Principle	Normative statement
TA-P01	Evidence before analytics	Architecture SHALL preserve original source/evidence and provenance before enrichment or analytical transformation.
TA-P02	Uncertainty preservation	Confidence, contradiction, inference class, and unknowns SHALL be first-class metadata.
TA-P03	Human accountability	A human SHALL remain accountable for case opening, high-impact assessment, dissemination, and closure.
TA-P04	Least privilege	Access SHALL be granted to the minimum data and actions required for a defined role and case purpose.
TA-P05	Data minimisation	The system SHALL support collection limitation, field-level classification, retention, and deletion workflows.
TA-P06	Reversible analysis	Merges, links, classifications, and analytical conclusions SHALL be reversible or supersedable without deleting historical state.
TA-P07	Traceability	Every transformation that materially affects an intelligence conclusion SHALL be auditable.
TA-P08	Separation of planes	Evidence storage, analytical processing, search, presentation, and dissemination SHALL be separable trust functions even when deployed on one host.
TA-P09	Secure by default	Default configuration SHALL deny unnecessary access, external exposure, bulk export, and privileged automation.
TA-P10	Portability	Canonical data SHALL be exportable in documented, machine-readable formats without vendor lock-in.
TA-P11	Graceful degradation	Loss of optional analytics SHALL not make evidence inaccessible or corrupt canonical records.
TA-P12	No surveillance maturity fallacy	Higher technical maturity SHALL mean stronger quality, security, and accountability, not broader collection.

3. Stakeholders, Roles, and Trust Boundaries

3.1 Human roles

Role	Core authority	Sensitive actions
Analyst	Create and analyse case-scoped information	Propose entity merge, typology match, hypothesis, assessment
Case Owner	Accountable for purpose and scope	Approve scope changes, pause/close case
Reviewer	Independent analytical review	Challenge evidence sufficiency and confidence
Publication/Referral Approver	Approve external dissemination	Authorize high-impact exports
Source Protector	Manage protected-source identity	Control reveal/re-identification
System Administrator	Operate platform	Infrastructure administration, not analytical approval
Security Administrator	Security policies, keys, incident response	Privileged security configuration
Auditor	Inspect control evidence	Read audit evidence without altering case state

3.2 Trust boundaries



A single-server deployment MAY host all zones on one machine, but the application SHALL preserve logical separation through roles, authorization policy, service boundaries, object classes, audit events, and export controls.

4. Capability Architecture

Technology Architecture capabilities use the namespace TA-CAP-xx (v0.1 used CAP-xx; order and meaning are unchanged). The product capability registry CAP-01...CAP-15 in the Product & Feature Specification v0.1.1 §7 is authoritative for product capabilities; the crosswalk in §4.1 maps between them. [v0.1.1 · A07]

ID	Capability	Minimum responsibility
TA-CAP-01	Case & workflow	Intake, charter, gates, tasks, review, closure
TA-CAP-02	Source & evidence	Acquisition, registration, hashing, extracts, provenance
TA-CAP-03	Entity & identity	Entity creation, aliases, external identifiers, resolution
TA-CAP-04	Relationships & ownership	Ownership, control, role, kinship/association, provenance
TA-CAP-05	Assets & events	Asset registry, event timeline, valuation metadata
TA-CAP-06	Value flow	Direct/documented/reconstructed/hypothetical value flows
TA-CAP-07	Graph analysis	Network traversal, communities, paths, shared attributes
TA-CAP-08	Search & discovery	Full text, fielded search, faceting, relationship search
TA-CAP-09	Typology analysis	Catalogue, indicators, matching, analytical cautions
TA-CAP-10	Hypothesis & assessment	Competing hypotheses, support/contradiction, gaps, confidence
TA-CAP-11	Intelligence products	Structured reports, redaction, review, dissemination
TA-CAP-12	Privacy & protection	Classification, minimisation, retention, source protection

ID	Capability	Minimum responsibility
TA-CAP-13	Security & IAM	Identity, authorization, secrets, secure administration
TA-CAP-14	Audit & assurance	Immutable audit events, control evidence, review history
TA-CAP-15	Integration	APIs, imports, exports, connectors, interoperability
TA-CAP-16	Operations	Observability, backup, DR, patching, health monitoring

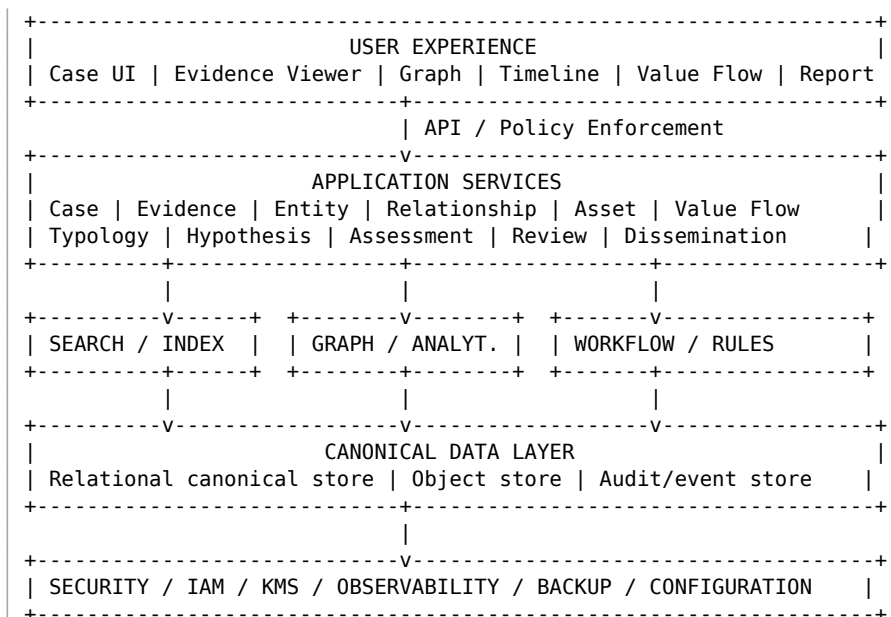
4.1 Crosswalk TA-CAP ↔ product CAP [v0.1.1 · A07]

Primary mapping first; secondary mappings in parentheses. Product capability names are from the Product & Feature Specification v0.1.1 §7.

TA-CAP	Technology capability	Product capability (CAP)
TA-CAP-01	Case & workflow	CAP-01 Case & Workflow
TA-CAP-02	Source & evidence	CAP-02 Source & Evidence (CAP-03 Document Intelligence for upload/extraction)
TA-CAP-03	Entity & identity	CAP-04 Entity & Identity
TA-CAP-04	Relationships & ownership	CAP-05 Relationship / Ownership / Asset
TA-CAP-05	Assets & events	CAP-05 Relationship / Ownership / Asset (assets); CAP-06 Timeline & Events (events)
TA-CAP-06	Value flow	CAP-07 Value-Flow
TA-CAP-07	Graph analysis	CAP-10 Search / Graph / Analytics
TA-CAP-08	Search & discovery	CAP-10 Search / Graph / Analytics
TA-CAP-09	Typology analysis	CAP-08 Typology & Indicator
TA-CAP-10	Hypothesis & assessment	CAP-09 Hypothesis & Assessment
TA-CAP-11	Intelligence products	CAP-11 Intelligence Products & Review (CAP-12 Dissemination & Referral)
TA-CAP-12	Privacy & protection	CAP-13 Administration & Governance (retention, policies); CAP-14 Security / Audit / Operations (classification enforcement, source protection)
TA-CAP-13	Security & IAM	CAP-14 Security / Audit / Operations
TA-CAP-14	Audit & assurance	CAP-14 Security / Audit / Operations
TA-CAP-15	Integration	CAP-15 Integrations & Automation (CAP-12 Dissemination & Referral for exports)
TA-CAP-16	Operations	CAP-14 Security / Audit / Operations

Product CAP-03 Document Intelligence and CAP-12 Dissemination & Referral have no dedicated TA-CAP; they are covered by TA-CAP-02 and TA-CAP-11/TA-CAP-15 respectively.

5. Logical Architecture



5.1 Architectural rule: canonical store vs projections

The architecture SHALL distinguish canonical records from derived projections. Search indexes, graph stores, caches, embeddings, materialized views, and analytics outputs MAY be rebuilt. Canonical evidence, provenance, decisions, and audit history SHALL NOT depend on a derived projection as the only source of truth.

Canonical-source rule

If a graph database or search index is lost, the system should be able to rebuild it from canonical objects. If the canonical evidence record is lost, the investigation may no longer be defensible.

6. Information and Data Architecture

6.1 Data planes

Plane	Contents	Persistence rule
Evidence plane	Original files, snapshots, extracts, hashes, acquisition metadata	Strong preservation; immutable original preferred
Canonical analytical plane	Case, source, evidence metadata, entity, relationship, asset, event, value flow, indicator, hypothesis, assessment	Versioned system of record
Projection plane	Search index, graph projection, vector index, cached analytics	Rebuildable
Operational plane	Jobs, queues, session state, cache	Ephemeral or recoverable
Audit plane	Authentication, object changes, approvals, exports, privileged actions	Append-oriented, access-restricted
Dissemination plane	Approved reports, exports, referral packages	Versioned, approval-bound

6.2 Storage responsibilities

Store class	Normative responsibility	Typical implementation (non-normative)
Relational store	Canonical structured objects, integrity constraints, workflow state	PostgreSQL / equivalent
Object store	Evidence files, archived source captures, generated products	S3-compatible (versioning/object-lock-capable; product selected per ADR, see Technical Stack v0.1.1 ADR-0005) / encrypted filesystem [v0.1.1 · A02]
Graph store	Relationship traversal and network analytics projection	Neo4j / Memgraph / graph extension / relational graph queries
Search index	Full-text and faceted retrieval	OpenSearch / Elasticsearch / PostgreSQL FTS
Audit/event store	Tamper-evident security and business audit events	Append-only database/table, WORM-capable store
Cache/queue	Async ingestion and analytics jobs	Valkey (Redis-protocol compatible) / message broker [v0.1.1 · A03]

6.3 Object identity and referential integrity

- Canonical object IDs SHALL be stable and opaque.
- External identifiers SHALL be represented separately from internal IDs.
- Deletion SHALL not silently orphan provenance or analytical dependencies.
- Relationships SHALL refer to canonical entity IDs, not display labels.
- Derived indexes SHALL carry canonical object IDs so analysts can navigate back to source records.

7. Case and Workflow Architecture

Workflow technology SHALL implement the methodology gates without turning them into a purely bureaucratic click path. Gates exist to ensure purpose, legality, evidentiary sufficiency, review, and controlled dissemination.

Gate	Trigger	Required system evidence
G0 Intake legitimacy	Before case creation	Purpose, source of lead, initial risk decision
G1 Charter approval	Before expanded collection	Question, scope, owner, time period, jurisdiction
G2 Collection authorization	Before higher-risk collection	Collection plan, legal/ethical review as applicable
G3 Analytical sufficiency	Before formal assessment	Evidence links, unresolved gaps, competing hypothesis record
G4 Peer review	Before high-impact conclusion	Reviewer identity, findings, response to challenge
G5 Dissemination approval	Before external export	Audience, minimisation/redaction, approver, product version
G6 Closure/reopen	At closure or reopen	Closure rationale, retention decision, reopen trigger

7.1 Workflow engine requirements

- State transitions SHALL be explicit and auditable.
- Privileged transitions SHOULD support four-eyes approval where impact is high.
- System SHALL support pause/hold without forcing analytical closure.
- A task engine MAY be used but task completion SHALL NOT itself imply analytical sufficiency.

8. Evidence and Provenance Architecture

8.1 Evidence lifecycle

| ACQUIRE -> REGISTER -> HASH -> CLASSIFY -> PRESERVE ORIGINAL -> EXTRACT -> VERIFY -> CITE -> REVIEW -> RETAIN/DISPOSE

8.2 Original evidence requirements

- Original acquired content SHALL be preserved separately from analyst annotations and transformed derivatives.
- Cryptographic digest SHOULD be computed at registration for files or snapshots where technically feasible.
- Timestamp, acquisition method, collector, source URL/location, and access context SHALL be recorded.
- Evidence extracts SHALL point to a precise location in the source where possible (page, paragraph, timestamp, field, image region).
- The system SHALL preserve a distinction between source availability and evidentiary reliability.

8.3 Derivative chain

Object	May derive from	Must retain link to
EvidenceExtract	Evidence	Evidence + source locator
Claim	EvidenceExtract / Evidence	Supporting evidence
Fact	Supported by evidence (mandatory) and, optionally, claim(s) (a separate object; the claims are not converted or altered) [v0.1.1 · C02]	Supporting claims (supporting_claim_refs) + evidence basis + VerificationDecision record + fact_status (PROVISIONAL/ESTABLISHED/DISPUTED/SUPERSEDED) [v0.1.1 · A10]
Indicator	Fact / event / relationship / value flow	Underlying facts

Object	May derive from	Must retain link to
Hypothesis	Indicators / facts	Support and contradiction
Assessment	Hypotheses / gaps	Reasoning basis + reviewer
IntelligenceProduct	Assessment + selected evidence	Approved assessment version

9. Entity Resolution Architecture

Entity resolution is a high-impact analytical capability because a false merge can create relationships that never existed, while a missed merge can conceal a meaningful network. Architecture SHALL make resolution decisions explicit and reversible.

9.1 Resolution pipeline

```

RECORDS / OBSERVATIONS
|
normalization
v
candidate generation
v
feature comparison
v
analyst review / rule decision
v
MERGE / POSSIBLE_MATCH / KEEP_SEPARATE / DEFER (UNMERGE reverses a MERGE)
v
canonical entity + ResolutionDecision record (append-only; entity resolution_status derived from it)

```

[v0.1.1 · ER] The decision step uses the ResolutionDecision values; tag moved out of the diagram.
[v0.1.1 · C19]

9.2 Mandatory controls

- No automatic merge solely on name similarity.
- Resolution SHALL store compared identifiers/features, confidence, decision maker, date, and evidence as an append-only ResolutionDecision (Data Model v0.1.1 §8.4); entity resolution state SHALL change only through such decisions. Legacy outcome names: SAME → MERGE; POSSIBLE SAME → POSSIBLE_MATCH; DISTINCT → KEEP_SEPARATE; UNRESOLVED → DEFER.
[v0.1.1 · ER]
- Merge SHALL be reversible without deleting original records.
- System SHOULD support alias, transliteration, former name, and spelling variants.
- High-impact entities SHOULD require stricter merge threshold or review.

9.3 Matching features

Feature family	Examples	Caution
Strong identifiers	Registration number, tax/company identifier, verified account/wallet ID	Identifier reuse or data entry error remains possible
Personal identifiers	DOB, nationality, occupation, address	Personal data handling restrictions
Contact	Phone, email, domain	Shared corporate contact can mislead
Location	Registered address, property address	Shared offices and virtual addresses common
Network	Common director, owner, device, intermediary	Correlation is not identity
Name	Exact/fuzzy/transliteration	Weak alone; language-sensitive

10. Knowledge Graph Architecture

The graph layer represents relationships for exploration and analytics. It SHALL not silently convert weak associations into factual relationships.

10.1 Graph representation

```
(Person) - [DIRECTOR_OF] -> (Company)
(Company) - [CONTRACTED_BY] -> (Agency)
(Company) - [OWNS] -> (Asset)
(Person) - [BENEFICIAL_OWNER_OF] -> (Company)
(Company) - [SHARES_ADDRESS_WITH] -> (Company)
```

10.2 Edge semantics

- Each material edge SHALL carry provenance or resolve to a canonical Relationship object.
- Edge type SHALL use controlled vocabulary.
- Confidence and relationship status SHALL be queryable.
- Temporal validity SHOULD be represented so expired relationships are not displayed as current by default.
- Analytical edges (e.g., POSSIBLY_CONTROLS) SHALL be visually distinct from verified edges.

10.3 Supported graph operations

Operation	Use	Guardrail
1-N traversal	Explore direct/indirect relationships	Default depth limits
Shortest path	Find connection between entities	Path ≠ causal relationship
Shared attribute query	Common address/phone/director	Shared attribute ≠ control
Community detection	Identify dense clusters	Exploratory only unless corroborated
Centrality	Find structurally central nodes	Importance ≠ culpability
Temporal graph filter	View relationships by period	Requires valid-time metadata

11. Search and Retrieval Architecture

- Search SHALL respect authorization at query and result-rendering time.
- Indexes containing sensitive content SHALL be treated as sensitive copies, not merely technical metadata.
- Full-text search SHOULD support language-aware tokenization where feasible.
- Search result snippets SHALL not reveal content the user is unauthorized to open.
- Bulk export/search APIs SHOULD be rate-limited and auditable.

11.1 Search modes

Mode	Purpose	Examples
Exact identifier	Deterministic lookup	Company number, case ID, wallet address
Fielded search	Structured discovery	Director name + date range
Full text	Document and note discovery	Contract wording, aliases
Graph-aware	Relationship-based search	Companies sharing an address
Similarity	Candidate discovery	Near-duplicate names or documents
Semantic/vector (optional)	Conceptual retrieval	Find documents discussing hidden ownership

Vector-search caution

Embeddings are derived representations. They SHALL NOT become the only copy of source content, and semantic similarity SHALL NOT be treated as evidence of factual relationship.

12. Timeline, Asset, and Value-Flow Analytics

12.1 Timeline service

The timeline service SHALL combine Events from multiple source types while retaining event type, source, confidence, and date precision. Approximate dates SHALL remain approximate.

12.2 Asset analysis

- Legal ownership, beneficial ownership, control, possession/use, and valuation SHALL be separate concepts.
- Historical ownership SHOULD be queryable.
- Estimated value SHALL include valuation date, currency, method, and confidence.
- Asset association SHALL not be displayed as ownership unless the evidence supports ownership.

12.3 Value-flow engine

Flow class	Meaning	Visualization rule
DIRECT	Transaction or transfer directly evidenced	Solid line; evidence link required
DOCUMENTED	Economic transfer documented but not necessarily bank-level transaction	Solid or distinct documented style
RECONSTRUCTED	Analyst reconstruction from contracts/assets/events	Dashed or otherwise distinct
HYPOTHETICAL	Possible flow used to test a hypothesis	Dotted; never included as established fact

Wire values of flow_class in storage, APIs and exports are the UPPER_SNAKE_CASE values above (DIRECT, DOCUMENTED, RECONSTRUCTED, HYPOTHETICAL), derived from the Data Model Annex A registry; display labels are separate and translatable. [v0.1.1 · A09]

12.4 Flow composition

```
Contract Award --DOCUMENTED--> Company A
Company A --RECONSTRUCTED--> Company B
Company B --DOCUMENTED ASSET PURCHASE--> Property P
```

```
System MUST NOT collapse this chain into:
Government --DIRECT PAYMENT--> Property P
```

13. Typology and Analytical Services

Typology technology supports structured comparison with known mechanisms. It SHALL assist analysts; it SHALL NOT automatically label a person or entity as a money launderer.

13.1 Typology service model

- Catalogue entries SHALL be versioned.
- Indicator logic SHOULD distinguish mechanism-specific, corroborating, contextual, disconfirming, and intelligence-gap indicators.

- A typology match SHALL retain the catalogue version used.
- Automated suggestions SHALL display the evidence and rule/features that produced the suggestion.
- Analyst rejection or modification SHALL be retained for learning and audit.

13.2 Analytical jobs

Job	Examples	Output status
Rule query	Rapid pass-through within defined window	Candidate indicator
Graph pattern	Circular ownership / shared-controller cluster	Analytical lead
Statistical anomaly	Unusual asset growth relative to known baseline	Lead; not a fact
Document extraction	Named entities, amounts, dates	Unverified observation until checked
Typology suggestion	Pattern resembles nominee ownership	Suggested match requiring analyst review

14. Hypothesis, Assessment, and Intelligence Product Services

14.1 Hypothesis engine

- System SHALL support multiple competing hypotheses.
- Evidence MAY support, contradict, or be neutral to a hypothesis.
- Unknowns SHALL be representable as IntelligenceGap objects.
- Hypothesis status SHALL not be derived only from a numeric score.
- System SHOULD make disconfirming evidence visible near supporting evidence.

14.2 Assessment service

Assessment authoring SHALL provide direct access to cited evidence, confidence rationale, intelligence gaps, alternative explanations, peer-review findings, and previous versions.

Confidence levels use the wire values HIGH, MODERATE, LOW, INSUFFICIENT_BASIS, with mandatory rationale for every level. INSUFFICIENT_BASIS (a judgement was attempted but the evidential basis is insufficient) is not a level below LOW and SHALL NOT be converted to LOW, null, zero, or omitted by any service, projection, search index or export. [v0.1.1 · A09]

14.3 Product generation

- Generated products SHALL be versioned and linked to the assessment snapshot used.
- Redaction/minimisation SHALL occur before dissemination.
- Export SHALL record recipient/audience, purpose, approver, timestamp, and product version.
- Publication-oriented outputs SHOULD support separation between internal evidentiary notes and public text.

15. API and Integration Architecture

15.1 API principles

- APIs SHALL enforce the same authorization semantics as the user interface.

- Object APIs SHOULD be resource-oriented and versioned.
- Write operations SHALL generate audit events.
- Bulk endpoints SHALL require explicit permission and rate control.
- Exports SHALL be schema-versioned.
- API clients SHALL receive stable IDs rather than relying on display names.

15.2 Minimum interfaces

Interface	Purpose	Notes
Case API	Create/read/update workflow metadata	Gate transitions separately privileged
Evidence API	Register metadata and retrieve authorized evidence	Original content may use signed/short-lived retrieval
Entity API	Entity CRUD and external IDs	Merge/split via dedicated decision endpoint; each creates a ResolutionDecision [v0.1.1 · ER]
Relationship API	Create/version relationships	Provenance required
Graph Query API	Bounded graph traversal	Read-only analytical service preferred
Search API	Authorized search	Security trimming mandatory
Product API	Generate/approve/export product	Approval and export audit
Admin API	Configuration and policy	Separate administrative privilege

15.3 Import and connector model

Connectors SHOULD land imported content in a staging/acquisition zone. Imported data SHALL be registered, classified, and mapped before becoming canonical. A connector failure SHALL NOT create partially trusted canonical objects without provenance.

16. Identity, Authentication, and Authorization

The architecture SHOULD apply zero-trust principles: no implicit trust based solely on network location, organisational ownership, or prior session context. Authentication and authorization SHALL be evaluated for access to protected resources.

16.1 Authentication

- MFA SHALL be supported for privileged and sensitive deployments.
- Phishing-resistant MFA SHOULD be used for administrators and high-impact approvers where feasible.
- Service identities SHALL be distinct from human identities.
- Shared analyst accounts SHALL NOT be used.
- Recovery processes SHALL be documented and auditable.
- In the MVP reference profile, browser authentication is a server-side session (BFF): the application is a confidential OIDC client (Authorization Code + PKCE); access/refresh/ID tokens stay server-side and the browser holds only an HttpOnly session cookie, with CSRF protection on unsafe methods (API Specification v0.1.1 §4). [v0.1.1 · A11]

16.2 Authorization model

```

ALLOW = Role Permission
        AND Case Membership / Purpose
        AND Object Classification
        AND Action Sensitivity
        AND Context Policy
        AND No explicit deny

```

16.3 Authorization dimensions

Dimension	Examples
Role	Analyst, Reviewer, Approver, Admin, Auditor
Case scope	Member of Case A but not Case B
Object class	Source identity, raw evidence, public source, assessment
Sensitivity	Public, Internal, Sensitive, Restricted, Source-protected (wire: PUBLIC ... SOURCE_PROTECTED, §17.1) [v0.1.1 · A08]
Action	View, annotate, link, merge, export, approve, administer
Purpose/ context	Active case purpose, break-glass emergency, legal hold

16.4 Break-glass access

Emergency access MAY exist but SHALL require explicit reason, stronger authentication where feasible, short duration, enhanced logging, and post-event review.

17. Privacy and Data-Protection Architecture

- Privacy metadata SHALL travel with objects or fields where required.
- System SHALL support data minimisation at collection, display, export, and retention stages.
- Sensitive personal data SHOULD be masked from users who do not need it.
- Protected-source identity SHOULD be isolated from ordinary analytical identifiers.
- Retention rules SHALL support legal hold and approved exceptions.
- Deletion/anonymisation SHALL preserve audit evidence necessary to explain that a controlled disposal occurred.

17.1 Classification model [v0.1.1 · A08]

The five-level model of the Data Model Specification v0.1.1 §16 is authoritative. Wire values, ordered least → most restrictive, are PUBLIC, INTERNAL, SENSITIVE, RESTRICTED, SOURCE_PROTECTED (display labels Public, Internal, Sensitive, Restricted, Source-protected; v0.1 wrote SOURCE-PROTECTED). Access labels (purpose, jurisdiction, embargo, legal-review, compartment, etc.) are additive restrictions; the most restrictive applicable level plus all labels apply. Derived objects and exports inherit the highest classification of their inputs unless a recorded reviewer downgrade decision exists. Unknown or missing classification fails closed (deny access, flag for classification).

Class	Example	Default treatment
PUBLIC	Published corporate registry	Normal case authorization
INTERNAL	Analyst notes	Case members only
SENSITIVE	Personal financial context, private correspondence lawfully held	Need-to-know; export restricted
RESTRICTED	High-risk identity data, unpublished allegation	Strong access restrictions
SOURCE_PROTECTED	Whistleblower identity or contact channel	Dedicated policy, minimal disclosure [v0.1.1 · A08]

18. Cryptography, Secrets, and Key Management

- Transport encryption SHALL protect remote access and service-to-service traffic crossing untrusted networks.

- Sensitive data at rest SHOULD be encrypted using platform-appropriate controls.
- Secrets SHALL not be stored in source code or ordinary configuration repositories.
- Key rotation and recovery SHALL be documented.
- Backup encryption keys SHALL be protected separately from backups.
- Cryptographic evidence hashes SHALL be distinguished from encryption keys and authentication secrets.

19. Security Architecture and Threat Model

19.1 High-value assets

Asset	Why valuable
Protected-source identity	Exposure can cause physical, legal, or reputational harm
Raw evidence	May contain sensitive personal or investigative information
Entity/relationship graph	Reveals investigation scope and associations
Assessment drafts	Can expose allegations before review
Credentials/tokens	Enable lateral access
Audit logs	Reveal activity and can prove misuse
Export packages	May contain concentrated high-impact intelligence

19.2 Threat scenarios

Threat	Examples
External compromise	Credential theft, application exploit, exposed storage
Malicious insider	Unauthorized browsing, export, source exposure
Compelled or coerced access	Legal/physical pressure on operator or host
Supply-chain compromise	Malicious dependency, build artifact, update channel
Data poisoning	False records introduced to manipulate analysis
Inference amplification	Weak signals algorithmically promoted to strong conclusions
Availability attack	Ransomware, deletion, denial of service
Metadata leakage	Backups, logs, search indexes, notifications expose case existence

19.3 Security control domains

- Secure configuration and patch management
- Application security verification
- Network segmentation and restricted administrative exposure
- Endpoint protection for administrator/analyst devices where in scope
- Rate limiting and anti-automation abuse
- File-type validation and malware scanning for uploads
- Content Security Policy and browser hardening
- Database and object-store least privilege
- Security incident detection and response
- Regular restore testing and credential/key rotation

20. Audit, Logging, and Non-Repudiation

20.1 Mandatory audit events

Event family	Examples
Authentication	Login success/failure, MFA change, recovery
Authorization	Denied sensitive action, break-glass
Case governance	Open, owner change, gate approval, close/reopen
Evidence	Register, hash, view/download sensitive item, delete/dispose
Analytical change	Entity merge/split, relationship change, hypothesis status, assessment version
Dissemination	Preview, approve, export, recipient/purpose
Administration	Role changes, policy/config changes, key/secrets operations
Integration	Connector import, bulk API use, failed mapping

20.2 Audit integrity

- Application users SHALL NOT be able to silently modify historical audit events.
- Audit access SHALL be restricted because logs may themselves contain sensitive metadata.
- Clock synchronization SHOULD be maintained across components.
- Audit retention SHALL be longer than transient operational logs where risk requires.
- High-impact events SHOULD be exportable to an independent monitoring or archival system.

21. Observability and Operational Monitoring

Operational observability SHOULD distinguish application telemetry from investigative content. Telemetry collection SHALL avoid unnecessary capture of evidence text, source identities, search queries, or document contents.

21.1 Signals

Signal	Examples	Privacy rule
Metrics	Request latency, job queue depth, storage use, error rate	No case content in labels
Logs	Service errors, security events, connector failures	Structured and minimised
Traces	Cross-service latency and failures	Avoid sensitive payloads
Health checks	Database, object store, index, queue	No sensitive data
Business controls	Pending review age, failed backup test	Aggregate when possible

OpenTelemetry or equivalent vendor-neutral instrumentation MAY be used; the normative requirement is consistent, privacy-aware telemetry that supports troubleshooting and assurance.

22. Reliability, Backup, and Disaster Recovery

22.1 Data criticality

Class	Examples	Recovery priority
Tier 0	Keys/secrets necessary to decrypt backups	Critical
Tier 1	Canonical database, evidence object store, audit store	Highest
Tier 2	Approved intelligence products, configuration	High
Tier 3	Search/graph indexes	Rebuildable

Class	Examples	Recovery priority
Tier 4	Caches, transient jobs	Lowest

22.2 Backup requirements

- Canonical data and evidence SHALL be backed up according to documented RPO/RTO targets.
- At least one backup copy SHOULD be logically isolated from the primary environment.
- Restore tests SHALL be performed periodically; a successful backup job is not proof of recoverability.
- Backups SHALL inherit sensitivity protections.
- Deletion/retention policy SHALL address backup copies, including realistic delayed deletion where immediate purge is technically infeasible.

22.3 Example recovery profiles

Profile	Indicative RPO	Indicative RTO	Suitable context
Small CSO	24 hours	1-3 days	Low transaction volume, single team
Operational	4 hours	8-24 hours	Active investigations, multiple analysts
High resilience	<=1 hour	<=4 hours	Time-sensitive/high-risk investigations

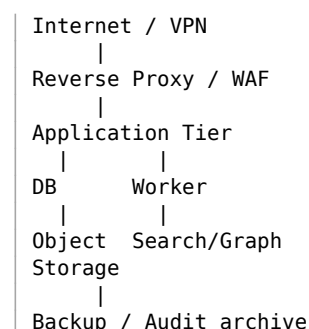
Values above are reference examples, not normative universal targets. Organisations SHALL set targets based on harm, resources, and operational need.

23. Deployment Models

23.1 Model A - Single-node secure deployment

Suitable for small organisations. Application, database, search/graph projections, and object storage may run on one hardened server or small virtual environment. Logical separation, backups, and least privilege remain required.

23.2 Model B - Segmented self-hosted deployment



23.3 Model C - Managed cloud deployment

Managed databases/object stores MAY be used where legal, risk, contractual, and threat-model considerations are acceptable. Provider access, encryption, location, backup, identity federation, and incident response responsibilities SHALL be documented.

23.4 Model D - High-risk isolated deployment

For especially sensitive work, organisations MAY use isolated or tightly controlled environments, separate source-protection stores, restricted egress, hardened analyst access, and dedicated export workflows.

24. Multi-Tenancy and Federation

24.1 Multi-tenancy

- Tenant isolation SHALL be explicit at authorization and data-access layers.
- Cross-tenant search SHALL be disabled by default.
- Shared infrastructure SHALL not imply shared analytical visibility.
- Administrative access to multiple tenants SHALL be separately privileged and audited.

24.2 Federation

Federation allows organisations to share selected intelligence without centralising all raw evidence. CS-AML federation SHOULD exchange explicitly approved objects or products, not provide unrestricted remote graph access.

```
Org A Canonical Store --approved package--> Exchange Layer --approved import--> Org B
raw evidence stays local unless specifically authorized
```

24.3 Federation package requirements

- Origin organisation/node identifier
- Schema version
- Object IDs and provenance
- Sharing classification and permitted use
- Confidence/uncertainty metadata
- Redaction/minimisation state
- Integrity signature or equivalent verification where feasible

25. Software Supply Chain and Secure Delivery

- Dependencies SHALL be inventoried and routinely updated.
- Build and release processes SHOULD produce provenance/attestation appropriate to organisational maturity.
- Production deployments SHOULD use reproducible or controlled build artifacts rather than ad-hoc developer workstations.
- Secrets SHALL be injected at deployment/runtime rather than embedded in artifacts.
- Security testing SHOULD include dependency, container/image, and application security checks.
- Critical updates SHALL have a documented emergency path.

OWASP ASVS MAY be used as a verification baseline for web application security controls. SLSA-style provenance and supply-chain levels MAY be used to strengthen source/build integrity. These are reference mappings, not requirements to obtain external certification.

26. AI and Automation Architecture

AI boundary

AI MAY assist extraction, summarisation, entity candidate generation, translation, search, pattern discovery, and drafting. AI SHALL NOT autonomously determine guilt, authorize dissemination, reveal protected-source identity, or silently overwrite analyst conclusions.

26.1 AI service boundary

- AI input scope SHALL be governed by data classification and provider/deployment risk.
- Sensitive cases SHOULD prefer locally controlled or contractually appropriate models where external processing is not acceptable.
- Prompts, retrieved context, outputs, and model/version metadata SHOULD be retained when AI output materially influences an assessment.
- AI-generated observations SHALL be marked as machine-generated until verified.
- Retrieval-augmented generation SHALL cite canonical objects/evidence where outputs are used analytically.
- Automated entity merges SHALL NOT occur without policy-defined validation/review.

26.2 Model governance

Concern	Required architectural response
Hallucination	Evidence-linked verification and analyst confirmation
Bias	Alternative hypotheses, review, evaluation datasets
Prompt/data leakage	Classification-aware routing, no unnecessary third-party submission
Model drift	Version capture and periodic evaluation
Automation bias	UI shows machine suggestion separately from analyst decision
Opaque score	Expose contributing features/rationale where possible

27. Performance and Scalability

CS-AML is analyst-centred rather than high-frequency payment processing. Architecture SHOULD optimize evidence retrieval, graph exploration, search responsiveness, and audit integrity rather than chase transaction-processing benchmarks irrelevant to civil-society investigation.

27.1 Performance classes

Workload	Target design concern
Interactive case navigation	Predictable page/API response
Full-text search	Sub-second to few-second typical query depending on corpus
Graph traversal	Bounded depth and result size; asynchronous for expensive analytics
Document processing	Asynchronous jobs with progress and retry
Bulk import	Staging, validation, backpressure
Report generation	Version-consistent snapshot; may run asynchronously

27.2 Scaling order

- Measure before scaling.

- Separate asynchronous workloads from interactive requests.
- Scale storage and indexes before introducing unnecessary service fragmentation.
- Partition by tenant/case only where justified by security or scale.
- Preserve canonical integrity when adding replicas, caches, and projections.

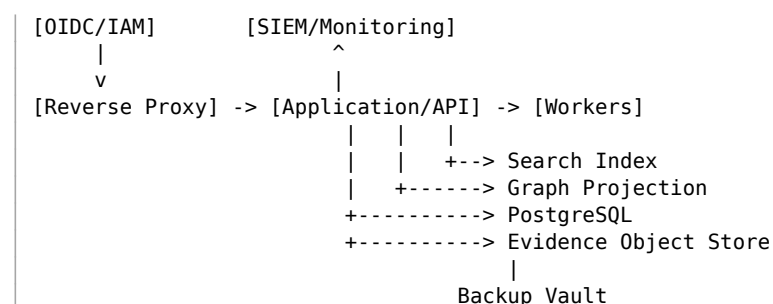
28. Reference Implementation Profiles

The following profiles are non-normative examples. Conformance is determined by capabilities and controls, not by using these technologies.

28.1 Profile R1 - Compact open-source deployment

Layer	Reference technology
Web/API	Django + Django REST Framework or equivalent
Canonical database	PostgreSQL
Spatial	PostGIS where geographic analysis is needed
Evidence storage	Encrypted filesystem or S3-compatible object storage (product and release line selected by ADR, with maintenance status, licence and restore evidence; MinIO Community is not a default — its upstream repository was archived on 25 April 2026) [v0.1.1 · A02]
Search	PostgreSQL FTS initially; OpenSearch when corpus/search needs grow
Graph	Relational graph queries initially; Neo4j/Memgraph optional projection
Async jobs	Celery/RQ + Valkey 8.x (Redis-protocol compatible, BSD-3-Clause; pinned release) or equivalent [v0.1.1 · A03]
IAM	Keycloak / OIDC provider; browser access via a server-side session (BFF) — tokens are not exposed to the browser [v0.1.1 · A11]
Reverse proxy	Nginx / Caddy
Observability	OpenTelemetry + Prometheus/Grafana or equivalent
Deployment	Containers or VMs on Linux

28.2 Profile R2 - Segmented production deployment



28.3 Technology selection criteria

- Self-hostability and exportability
- Security maintenance cadence
- Mature access controls
- Backup/restore tooling
- Data portability
- Operational skill availability

- Resource footprint
- Ability to preserve provenance and audit
- Avoidance of mandatory third-party exposure of sensitive data

29. Architecture Decision Records

Material architecture choices SHALL be documented so future maintainers understand why a decision was made, what risks were accepted, and when reconsideration is required.

ADR field	Requirement
Decision ID	Stable identifier
Context	Problem and constraints
Decision	Chosen approach
Alternatives	Material options considered
Security/privacy impact	Threat and data-protection consequences
Operational impact	Skills, cost, maintenance
Status	Proposed / Accepted / Superseded
Review trigger	Date, scale threshold, threat change, dependency change

30. Conformance and Verification

30.1 Technology conformance levels

Level	Description	Minimum expectation
T1 - Foundational	Small deployment with core records and controls	Case, evidence/provenance, entity/relationship, authorization, audit, backup, controlled export
T2 - Operational	Structured analytics and review	Search, timeline/value flow, workflow gates, stronger IAM, tested restore, product approval
T3 - Analytical	Advanced graph/typology/automation	Graph analytics, entity-resolution workflow, typology services, controlled AI/automation
T4 - Federated/High Assurance	Multi-organisation or high-risk operation	Federation controls, stronger isolation, independent audit archive, supply-chain provenance, enhanced DR

30.2 Mandatory verification questions

- Can every externally disseminated analytical conclusion be traced to an approved assessment and underlying evidence?
- Can the system distinguish direct evidence from reconstructed or hypothetical value flow everywhere it appears?
- Can entity merges be reviewed and reversed?
- Can a user search or export data outside their authorized case/purpose?
- Can protected-source identity be separated from ordinary analytical records?
- Can canonical records be restored independently of graph/search projections?
- Are high-impact actions attributable to named actors with timestamps and versions?
- Can the organisation demonstrate successful restore tests, not merely backup jobs?
- Can sensitive data be exported without an auditable approval event?
- Does any AI/automation path silently transform suggestion into fact or allegation?

30.3 Architecture evidence pack

- Current logical architecture diagram
- Data-flow and trust-boundary diagram
- Service/component inventory
- Data-store inventory and classification
- IAM/role matrix
- Network exposure inventory
- Backup/restore evidence
- Audit event catalogue
- Security hardening baseline
- Incident response integration
- Connector/integration register
- ADR register
- Conformance test results

Annex A. Normative Component Catalogue

ID	Component	Normative responsibility
TA-C01	Case Service	Canonical case metadata, ownership, scope, states
TA-C02	Evidence Service	Evidence registration, hashes, extracts, secure retrieval
TA-C03	Entity Service	Entities, aliases, external IDs, resolution decisions
TA-C04	Relationship Service	Versioned provenance-linked relationships
TA-C05	Asset/Event Service	Assets, events, temporal state
TA-C06	Value-Flow Service	Typed value flows and legs
TA-C07	Graph Projection	Rebuildable relationship analytics
TA-C08	Search Service	Authorized retrieval and indexing
TA-C09	Typology Service	Versioned typology/indicator support
TA-C10	Hypothesis/Assessment Service	Competing hypotheses, gaps, confidence, assessments
TA-C11	Review/Approval Service	Peer review and approval state
TA-C12	Product/Dissemination Service	Generation, minimisation, export log
TA-C13	IAM/Policy Enforcement	Authentication and authorization
TA-C14	Audit Service	Append-oriented audit events
TA-C15	Object/Evidence Store	Original and derivative binary content
TA-C16	Integration Gateway	Imports, connectors, external APIs
TA-C17	Job/Queue Service	Asynchronous processing
TA-C18	Observability Stack	Metrics/logs/traces with privacy constraints

Annex B. Minimum Network Exposure Baseline

- Database, graph, search, object storage, queue, and administrative ports SHOULD NOT be directly exposed to the public Internet.
- User-facing access SHOULD terminate at an authenticated application/reverse-proxy boundary.
- Administrative access SHOULD use a restricted management path such as VPN, zero-trust access proxy, bastion, or equivalent.
- Outbound egress SHOULD be limited where high-risk threat models justify it.
- Connector services needing external access SHOULD be isolated from evidence stores where practical.

Annex C. Architecture Review Checklist

- ☐ Canonical vs derived stores identified
- ☐ Trust boundaries documented
- ☐ All external interfaces inventoried
- ☐ IAM and role matrix approved
- ☐ Protected-source isolation designed
- ☐ Evidence hashing/preservation path tested
- ☐ Entity merge/split path tested
- ☐ Value-flow classes visible in UI/export
- ☐ Search authorization tested
- ☐ Bulk export controls tested
- ☐ Backup restore tested
- ☐ Audit tamper resistance reviewed
- ☐ AI data-routing reviewed
- ☐ Software supply-chain controls documented
- ☐ Disaster recovery exercise completed or scheduled

Annex D. Reference Standards and External Lineage

The CS-AML Technology Architecture is implementation-neutral. The following external references are informative anchors for specific technical control domains and do not supersede the CS-AML normative requirements:

Reference	Use in CS-AML architecture
NIST SP 800-207	Zero Trust Architecture; resource-focused access and removal of implicit trust based on network location.
NIST SP 800-207A	Cloud-native zero-trust access principles using user and service identities.
OWASP ASVS 5.0.0	Application-security verification requirements for web applications and supporting controls.
OpenTelemetry	Vendor-neutral telemetry framework for traces, metrics, and logs.
SLSA v1.2	Software supply-chain security levels, provenance, and source/build integrity concepts.

Annex E. Example End-to-End Technical Trace

- Analyst registers source URL/document
- Evidence Service stores original + hash + acquisition metadata
- Extract created for relevant paragraph/page
- Claim created and linked to extract
- Fact created as PROVISIONAL, supported by evidence and, optionally, the claim(s) (which remain unchanged); its CREATE
↪ VerificationDecision is written in the same transaction; ESTABLISHED only after review by someone other than the
↪ proposer
- Entity created / resolved to canonical identity
- Relationship created with evidence + confidence + valid time
- Graph projection updates asynchronously
- Value-flow reconstruction links contract, company, and asset
- Typology Service suggests possible nominee/layering pattern
- Analyst records competing hypothesis and intelligence gaps
- Assessment drafted with evidence citations and confidence
- Reviewer challenges assumptions; analyst revises
- Approver authorizes redacted intelligence product
- Dissemination Service records recipient, purpose, version, timestamp
- Audit Service preserves the decision and export history

[v0.1.1 · A10] Step 5 follows the Claim/Fact lifecycle (Data Model §7.4–7.6). [v0.1.1 · C01, C02] Tags moved out of the trace. [v0.1.1 · C19]

Document Status and Change Control

Version 0.1.1 is a proposed normative baseline (draft for review), not validated, intended for controlled implementation and field testing after review. [v0.1.1 · A01] Architecture changes that alter trust boundaries, canonical object semantics, dissemination controls, source protection, audit integrity, or the distinction between evidence and analytical inference **SHOULD** be treated as framework-level changes rather than ordinary implementation choices.